Reviewer Pack — Möbius Mass of Rectangle Poset Lower-Bounds D^cc

Entry 55903661e64a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 09:52:07 UTC

Möbius Mass of Rectangle Poset Lower-Bounds D^cc

Entry ID: `55903661e64a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 09:52:07 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Möbius-function theory / Rota's incidence-algebra of finite posets **Field B** (complexity object): Deterministic two-party communication complexity D^cc of explicit Boolean functions

Statement:

> Let $f:\{0,1\}^{n\times\{0,1\}^n}\to\{0,1\}$ and let M_f be its $2^n\times 2^n$ communication matrix. Define P_f as the finite poset whose elements are the maximal 1-monochromatic combinatorial rectangles $R=A\times B$ of M_f, ordered by componentwise inclusion ($A\times B \le A'\times B'$ iff $A\subseteq A'$ and $B\subseteq B'$), and let μ_f be the Möbius function on its incidence algebra. Conjecture: for every f with $D^{cc}(f) \ge 4$, the total Möbius mass $MM(f) := \Sigma_{R\le R' \text{ in } P_f} |\mu_f(R,R')|$ satisfies $\log_2 MM(f) \ge D^{cc}(f)/4$; a single f with $D^{cc}(f) \ge 4$ and $\log_2 MM(f) < D^{cc}(f)/4$ falsifies it.

Rationale (proposer's reasoning):

> D^cc(f) is governed entirely by the lattice of monochromatic rectangles, but standard lower bounds use only coarse cardinality features (cover/partition/fooling). The Möbius function detects how far this lattice is from a Boolean algebra: hard functions should generate many genuinely non-distributive intervals among maximal rectangles, producing exponential signed cancellation that mirrors the protocol-tree branching needed to separate them. Because MM is a relational invariant of the rectangle poset (not a single truth-table number) and is not preserved by ring/polynomial extension of the matrix entries, it avoids natural-proofs single-number traps and the algebrization route.

Taxonomy category: `COMM_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `6db141f64279a717`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across $n\in\{3,4,5,6,7,8\}$ × {DISJ,EQ,GT,IP,ANDOR, random p=0.25, random p=0.5} × 30 seeds, compute $\log_2 MM(f)$ and D^cc(f) exactly; conjecture is SUPPORTED iff ≥80% of

instances with D^cc(f)≥4 satisfy log_2 MM(f) ≥ D^cc(f)/4 AND no instance violates it; any single violation FALSIFIES.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Möbius function poset communication complexity rectangles` - `incidence algebra monochromatic rectangle cover lower bound` - `Rota Möbius combinatorial rectangle deterministic communication`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def f(x, y):
        return x == y

    M = communication_matrix(f)
    n = len(M)
    P_f = maximal_rectangles(M)
    mu_f = moebius_function(P_f)
    MM_f = total_moebius_mass(mu_f, P_f)
    D_cc_f = disjointness_communication_complexity(f)

    return {
        "metric_name": "log2_MM",
        "metric_value": math.log2(MM_f),
        "instances_tested": 1,
        "conjecture_holds": log2_MM >= D_cc_f / 4 if D_cc_f >= 4 else False,
        "counterexample": "" if (D_cc_f < 4 or log2_MM >= D_cc_f / 4) else f"Counterexample: D^cc(
    }

def communication_matrix(func):
    n = len(next(iter(func.values())))
```

```
    M = [[func((i, j), (x, y)) for x in range(n)] for i in range(n)]
    return M

def maximal_rectangles(M):
    def support(row):
        return [j for j, val in enumerate(row) if val == 1]

    P_f = []
    n = len(M)
    for A in range(1 << n):
        rows_A = [i for i in range(n) if (A & (1 << i)) != 0]
        B = set.intersection(*[set(support(M[i])) for i in rows_A])
        if B and all((A & (1 << i)) != 0 for i in rows_A):
            P_f.append((rows_A, list(B)))
    return P_f

def moebius_function(P_f):
    n = len(P_f)
    mu = [[0] * n for _ in range(n)]

    def dfs(i, j):
        if i == j:
            return 1
        if i > j:
            return 0
        mu[i][j] = -sum(mu[i][k] for k in range(i+1, j))
        return mu[i][j]

    for i in range(n):
        dfs(0, i)
    return mu

def total_moebius_mass(mu_f, P_f):
    MM_f = sum(abs(mu_f[i][j]) for i in range(len(P_f)) for j in range(i+1, len(P_f)))
    return MM_f

def disjointness_communication_complexity(func):
    n = len(next(iter(func.values())))
    def partition_number(submat):
        if not submat:
            return 0
        m = len(submat)
        dp = [[0] * (m + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(i, m + 1):
                dp[i][j] = dp[i-1][j] + sum(func((i-1, k), (x, y)) for x in range(n) for y in rang
        return dp[1][m]

    memo = {}
    def memoized_partition_number(submat):
        if not submat:
            return 0
        key = tuple(tuple(row) for row in submat)
        if key not in memo:
            memo[key] = partition_number(submat)
        return memo[key]
```

```
    def submatrix(A, B):
        return [[func((i, j), (x, y)) for x in A for y in B] for i in range(n) for j in range(n)]

    def all_subsets(s):
        subsets = []
        for r in range(1 << len(s)):
            subset = [s[i] for i in range(len(s)) if (r & (1 << i)) != 0]
            subsets.append(subset)
        return subsets

    max_partition_size = 0
    for A in all_subsets(range(n)):
        for B in all_subsets(range(n)):
            submat = submatrix(A, B)
            partition_size = memoized_partition_number(submat)
            if partition_size > max_partition_size:
                max_partition_size = partition_size

    return max_partition_size

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_log2_MM = sum(r["metric_value"] for r in results) / len(results)
    std_log2_MM = math.sqrt(sum((r["metric_value"] - mean_log2_MM)**2 for r in results) / len(resu
    support_fraction = sum(1 for r in results if "conjecture_holds" in r and r["conjecture_holds"]

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_log2_MM} std={std_log2_MM} support_fraction={support_
    elif any("counterexample" in r and r["counterexample"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if "counterexample" in r and r["count
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c3ade04.py", line 123, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c3ade04.py", line 24, in run_trial
    M = communication_matrix(f)
```

```
        ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c3ade04.py", line 40, in communication
    n = len(next(iter(func.values())))
                      ^^^^^^^^^^^
AttributeError: 'function' object has no attribute 'values'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with an AttributeError (the code called .values() on a function object) before any trials produced data, so neither the support nor falsification condition can be evaluated. | next: Fix communication_matrix to accept a callable f:$\{0,1\}^{n\times\{0,1\}}$n→{0,1} (infer n from the signature or pass n explicitly) and rerun the pre-registered sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	202405
2	preregistration	claude_max	opus	0	0	6679
3	novelty	claude_max	opus	0	0	3347
4	novelty	claude_max	opus	0	0	7058
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16010
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16194
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17168
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14675
9	judge	claude_max	opus	0	0	4086

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 287623 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/55903661e64a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/55903661e64a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/55903661e64a.tar.gz` (if generated)